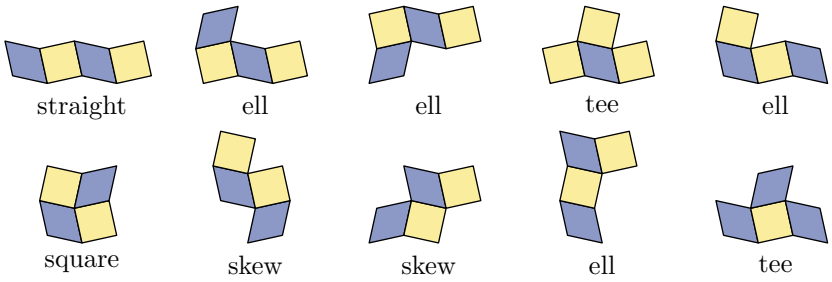
September 5, 2026 at 20:33

1. Introduction. Exercise 7.2.2.1–323 closes the survey of polyforms with *polyskews*, the shapes you get by joining squares alternately with rhombuses, in checkerboard fashion. It asks three things: (a) how to draw such skewed diagrams, (b) how to reduce polyskews to polyominoes, and (c) in how many ways the ten tetraskews make a skewed rectangle.

Answer 323 says the 4×10 frame has 486 solutions and the 5×8 frame 572; that there are 3648 ways to fit the pieces into a 2×21 frame, while 2×20 is too tight; that the counts can be halved because solutions come in dual pairs; that the 486 come from exactly 226 unskewed arrangements distinct under reflections, 17 of which yield two dual pairs; and that Michael Keller's problem of packing two 4×5 frames at once has just 24 solutions. This program checks all of that.

Everything comes out exactly but one number. The 2×21 frame has 72 solutions here, not 3648; and 3648 is the count for a 2×22 frame, which holds 44 cells and so leaves four of them empty rather than two. Nothing else in the exercise or the answer needs correcting.

Here are the ten tetraskews. Squares are drawn light and rhombuses dark, and the skew is exaggerated so that the two can be told apart at a glance.




```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strings"
    "time"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Types 4 ⟩
⟨ Functions 5 ⟩
func main( ) {
    ⟨ Read the command line 2 ⟩
    ⟨ Do what the mode asks 3 ⟩
}
```

2. The modes are all cheap; the whole lot runs in a couple of minutes. *pieces* grows the polyskews and checks the counts the exercise states; *pack* fills the frames; *dual* looks at the pairing of solutions and at the unskewed arrangements behind them; *pixel* redoes the packing through the reduction of part (b); and *tilings* comes at the same numbers from the other end, by skewing every arrangement of ten tetrominoes.

⟨ Read the command line 2 ⟩ ≡

```
mode := flag.String("mode", "all", "pieces, pack, dual, pixel, tilings, or all")
flag.Parse()
```

This code is used in section 1.

3. ⟨ Do what the mode asks 3 ⟩ ≡

```
if *mode ≡ "pieces" ∨ *mode ≡ "all" {
    ⟨ Count the polyskews 20 ⟩
}
if *mode ≡ "pack" ∨ *mode ≡ "all" {
    ⟨ Fill the frames 28 ⟩
}
if *mode ≡ "dual" ∨ *mode ≡ "all" {
    ⟨ Pair up the solutions 47 ⟩
}
if *mode ≡ "pixel" ∨ *mode ≡ "all" {
    ⟨ Fill the frames through the pixel reduction 40 ⟩
}
if *mode ≡ "tilings" ∨ *mode ≡ "all" {
    ⟨ Skew every arrangement of ten tetrominoes 58 ⟩
}
```

This code is used in section 1.

4. The skewed grid. Part (a) offers a coordinate system: skew the vertices (m, n) of the square grid to

$$(m, n) \mapsto (m - \epsilon[n \text{ odd}],\ n - \epsilon[m \text{ odd}]),$$

where ϵ is the degree of skew. Working out what that does to a cell settles everything else. Cell (x, y) has corners (x, y), $(x + 1, y)$, $(x + 1, y + 1)$, $(x, y + 1)$, and after the skew its two edge vectors are $(1, \pm\epsilon)$ and $(\mp\epsilon, 1)$: perpendicular when $x+y$ is odd, and not when it is even. So the cells with $x+y$ odd are the squares and the rest are the rhombuses, in checkerboard fashion, just as the exercise says.

The finer structure matters too. At (x, y) both even the rhombus is stretched along the direction $(1, -1)$, at both odd along $(1, 1)$; and the square spins counterclockwise when x is odd and y even, clockwise the other way round. That is the "clockwise or counterclockwise spin" the answer points out.

⟨ Types 4 ⟩ ≡

```
type cell struct { x, y int }
```

This code is also defined in sections 7, 12, 34, 41.

This code is used in section 1.

5. ⟨ Functions 5 ⟩ ≡

```
func mod2(v int) int { return ((v % 2) + 2) % 2 }
```

This code is also defined in sections 6, 8, 9, 13, 14, 16, 17, 19, 21, 22, 24, 27, 35, 36, 37, 38, 42, 43, 44, 45, 53, 54, 56.

This code is used in section 1.

6. *kindOf* returns 0 for a rhombus and 1 for a square, together with the lean of the rhombus or the spin of the square.

⟨ Functions 5 ⟩ +≡

```
func kindOf(c cell)(int, int) {
    a, b := mod2(c.x), mod2(c.y)
    switch {
    case a ≡ 0 ∧ b ≡ 0:
        return 0, 0
    case a ≡ 1 ∧ b ≡ 1:
        return 0, 1
    case a ≡ 1 ∧ b ≡ 0:
        return 1, 0
    }
    return 1, 1
}
```

7. The symmetries of the grid. A polyskew may be turned and flipped, but only in ways that carry the skewed grid to itself. The linear part must be one of the eight symmetries of the square, and a shift by $(2, 0)$ or $(0, 2)$ is harmless, so what is left to decide is which of the four shifts modulo 2 goes with each linear map. A map is allowed when it takes every cell to a cell of the right kind: rhombuses to rhombuses with the lean the map gives them, squares to squares with the spin reversed exactly when the map reverses orientation.

⟨ Types 4 ⟩ +≡

```
type sym struct {
    m [4]int
    t cell
}
```

8. ⟨ Functions 5 ⟩ +≡

```
var mats = [8][4]int{
    {1, 0, 0, 1}, {0, −1, 1, 0}, {−1, 0, 0, −1}, {0, 1, −1, 0},
    {1, 0, 0, −1}, {0, 1, 1, 0}, {−1, 0, 0, 1}, {0, −1, −1, 0},
}
func apply(m [4]int, c cell) cell {
    return cell{m[0] * c.x + m[1] * c.y, m[2] * c.x + m[3] * c.y}
}
func det(m [4]int) int { return m[0] * m[3] − m[1] * m[2] }
```

9. ⟨ Functions 5 ⟩ +≡

```
func symmetries( ) []sym {
    var out []sym
    for _, m := range mats {
        for tx := 0; tx < 2; tx++ {
            for ty := 0; ty < 2; ty++ {
                ⟨ Keep the map if it carries every kind of cell correctly 10 ⟩
            }
        }
    }
    return out
}
```

10. ⟨ Keep the map if it carries every kind of cell correctly 10 ⟩ ≡

```
ok := true
for x := 0; x < 2 ∧ ok; x++ {
    for y := 0; y < 2 ∧ ok; y++ {
        c := cell{x, y}
        k, o := kindOf(c)
        d := apply(m, c)
        k2, o2 := kindOf(cell{d.x + tx, d.y + ty})
        ⟨ Work out what the image of c ought to be 11 ⟩
        if k2 ≠ k ∨ o2 ≠ want {
            ok = false
        }
    }
}
if ok {
    out = append(out, sym{m, cell{tx, ty}})
}
```

This code is used in section 9.

11. A rhombus stretched along $(1, -1)$ goes to one stretched along the image of that direction, which is $(1, -1)$ or $(1, 1)$ again; a square's spin survives a rotation and is reversed by a reflection.

⟨ Work out what the image of c ought to be 11 ⟩ ≡

```
want := o
if k ≡ 0 {
    v := cell{1, -1}
    if o ≡ 1 {
        v = cell{1, 1}
    }
    if w := apply(m, v); w.x ≡ w.y {
        want = 1
    } else {
        want = 0
    }
} else if det(m) < 0 {
    want = 1 − o
}
```

This code is used in section 10.

12. Growing the polyskews. A polyskew is a set of cells, edge-connected, with the kinds alternating of their own accord because the grid alternates. Two of them are the same shape when a symmetry of the grid carries one to the other. Sliding a shape to a canonical corner is the one delicate step: only even shifts are symmetries, so a shape is slid until its least corner sits at 0 or 1 in each direction, not at 0.

⟨ Types 4 ⟩ +≡

```
type shape []cell
```

13. ⟨ Functions 5 ⟩ +≡

```
func floorDiv(a, b int) int {
    q := a/b
    if a % b ≠ 0 ∧ (a < 0) ≠ (b < 0) {
        q--
    }
    return q
}
```

14. ⟨ Functions 5 ⟩ +≡

```
func (s shape) norm() shape {
    t := append(shape{}, s...)
    ⟨ Slide the shape to its corner 15 ⟩
    sort.Slice(t, func(i, j int) bool {
        if t[i].y ≠ t[j].y {
            return t[i].y < t[j].y
        }
        return t[i].x < t[j].x
    })
    return t
}
```

15. ⟨ Slide the shape to its corner 15 ⟩ ≡

```
mx, my := t[0].x, t[0].y
for _, c := range t {
    if c.x < mx {
        mx = c.x
    }
    if c.y < my {
        my = c.y
    }
}
dx, dy := 2 * floorDiv(mx, 2), 2 * floorDiv(my, 2)
for i := range t {
    t[i].x -= dx
    t[i].y -= dy
}
```

This code is used in section 14.

16. ⟨ Functions 5 ⟩ +≡

```
func (s shape) key() string {
    var b strings.Builder
    for _, c := range s {
        fmt.Fprintf(&b, "%d,%d;", c.x, c.y)
    }
    return b.String()
}
func (s shape) images() []shape {
    seen := map[string]bool{}
    var out []shape
    for _, g := range symmetries() {
        t := make(shape, len(s))
        for i, c := range s {
            d := apply(g.m, c)
            t[i] = cell{d.x + g.t.x, d.y + g.t.y}
        }
        t = t.norm()
        if k := t.key(); ¬seen[k] {
            seen[k] = true
            out = append(out, t)
        }
    }
    return out
}
func (s shape) canon() string {
    best := ""
    for _, t := range s.images() {
        if k := t.key(); best ≡ "" ∨ k < best {
            best = k
        }
    }
    return best
}
```

17. Growing them one cell at a time should give two monoskews, one diskew, five triskews and ten tetraskews, which is what the exercise states. The two monoskews are the two kinds of cell, so the growth starts from both.

⟨ Functions 5 ⟩ +≡

```
func grow(upto int) [][]shape {
    all := make([][]shape, upto + 1)
    all[1] = []shape{{cell{0, 0}}, {cell{1, 0}}}
    for n := 2; n ≤ upto; n++ {
        seen := map[string]bool{}
        for _, s := range all[n − 1] {
            ⟨ Add one cell to s in every way 18 ⟩
        }
    }
    return all
}
```

18. ⟨ Add one cell to s in every way 18 ⟩ ≡

```
in := map[cell]bool{ }
for _, c := range s {
    in[c] = true
}
for _, c := range s {
    for _, d := range []cell {{c.x + 1, c.y}, {c.x − 1, c.y},
        {c.x, c.y + 1}, {c.x, c.y − 1} }{
            if in[d] {
                continue
            }
            t := append(append(shape{ }, s...), d).norm( )
            if k := t.canon( ); ¬seen[k] {
                seen[k] = true
                all[n] = append(all[n], t)
            }
        }
}
```

This code is used in section 17.

19. The dual. Skewing with $-\epsilon$ instead of ϵ gives the same tiling shifted: a short calculation with the formula of part (a) shows that $V_\epsilon(m+1, n+1) = V_{-\epsilon}(m, n) + (1-\epsilon, 1-\epsilon)$. So changing all the spins amounts to sliding a shape by $(1, 1)$, which is not a symmetry of the grid—even shifts are—and it turns every lean and every spin around. That is the *dual* of answer 323.

On the pieces it should fix four and swap the other six in three pairs, since the answer says $K \leftrightarrow L$, $S \leftrightarrow Z$ and $U \leftrightarrow V$ are exchanged and says nothing about the rest.

⟨ Functions 5 ⟩ +≡

```
func (s shape) dual() shape {
    t := make(shape, len(s))
    for i, c := range s {
        t[i] = cell{c.x + 1, c.y + 1}
    }
    return t.norm()
}
```

20. Forgetting the skew turns a polyskew into a polyomino, and answer 323 names the five tetromino shapes the ten tetraskews come from: one square, one straight, two skews, two tees and four ells. The dual leaves a piece alone exactly when the piece is not chiral, so counting the fixed ones is a way of seeing that list.

⟨ Count the polyskews 20 ⟩ ≡

```
all := grow(4)
want := []int{0, 2, 1, 5, 10}
for n := 1; n ≤ 4; n++ {
    mark := "ok"
    if len(all[n]) ≠ want[n] {
        mark = "MISMATCH"
    }
    fmt.Printf("%d␣cells:␣%2d␣polyskews␣(the␣exercise␣says␣%2d)␣␣%s\n",
        n, len(all[n]), want[n], mark)
}
fixed := 0
for _, s := range all[4] {
    if s.dual().canon() ≡ s.canon() {
        fixed++
    }
}
fmt.Printf("the␣dual␣fixes␣%d␣tetraskews␣and␣swaps␣the␣other␣%d␣in␣pairs\n",
    fixed, len(all[4]) − fixed)
```

This code is used in section 3.

21. Packing a frame. A frame is a rectangle of cells, and the exact cover problem has one item per cell and one per piece. When the frame has more cells than the ten pieces cover, the extra ones are left empty by a single item of multiplicity h—one item, not h of them, so that a set of empty cells is counted once rather than once for each way of ordering it.

⟨ Functions 5 ⟩ +≡

```
func rect(x0, y0, w, h int) map[cell]bool {
    f := map[cell]bool{}
    for y := y0; y < y0 + h; y++ {
        for x := x0; x < x0 + w; x++ {
            f[cell{x, y}] = true
        }
    }
    return f
}
func cname(c cell) string { return fmt.Sprintf("c%02d.%02d", c.x + 50, c.y + 50) }
```

22. Only even shifts are allowed, but the eight images of a piece already carry the odd shifts that its reflections need, so trying every even shift of every image reaches every placement exactly once.

⟨ Functions 5 ⟩ +≡

```
func placements(s shape, frame map[cell]bool) [][]cell {
    var out [][]cell
    seen := map[string]bool{}
    for _, im := range s.images() {
        for dy := -40; dy ≤ 40; dy += 2 {
            for dx := -40; dx ≤ 40; dx += 2 {
                ⟨ Keep the shifted image if it lies in the frame 23 ⟩
            }
        }
    }
    return out
}
```

23. ⟨ Keep the shifted image if it lies in the frame 23 ⟩ ≡

```
var p shape
ok := true
for _, c := range im {
    d := cell{c.x + dx, c.y + dy}
    if ¬frame[d] {
        ok = false
        break
    }
    p = append(p, d)
}
if ¬ok {
    continue
}
p = p.norm()
for i := range p {
    p[i] = cell{p[i].x + dx, p[i].y + dy}
}
sort.Slice(p, func(i, j int) bool {
    if p[i].y ≠ p[j].y {
        return p[i].y < p[j].y
    }
    return p[i].x < p[j].x
})
if k := p.key(); ¬seen[k] {
    seen[k] = true
    out = append(out, p)
}
```

This code is used in section 22.

24. ⟨ Functions 5 ⟩ +≡

```
func problem(frame map[cell]bool, ps []shape, holes int)(string, int) {
    var items []string
    for c := range frame {
        items = append(items, cname(c))
    }
    sort.Strings(items)
    var b strings.Builder
    for i := range ps {
        fmt.Fprintf(&b, "p%d␣", i)
    }
    if holes > 0 {
        fmt.Fprintf(&b, "%d:%d|hole␣", holes, holes)
    }
    b.WriteString(strings.Join(items, "␣"))
    b.WriteString("\n")
    n := 0
    ⟨ Write an option for every placement 25 ⟩
    ⟨ Write an option for every empty cell 26 ⟩
    return b.String(), n
}
```

25. ⟨ Write an option for every placement 25 ⟩ ≡

```
for i, s := range ps {
    for _, p := range placements(s, frame) {
        fmt.Fprintf(&b, "p%d", i)
        for _, c := range p {
            fmt.Fprintf(&b, " %s", cname(c))
        }
        b.WriteString("\n")
        n++
    }
}
```

This code is used in section 24.

26. ⟨ Write an option for every empty cell 26 ⟩ ≡

```
if holes > 0 {
    for _, c := range items {
        fmt.Fprintf(&b, "hole %s\n", c)
        n++
    }
}
```

This code is used in section 24.

27. Without holes this is an ordinary exact cover problem and the XCC engine solves it; with them the multiplicity needs the MCC engine.

⟨ Functions 5 ⟩ +≡

```
func solve(in string, holes int)(int, time.Duration) {
    t0 := time.Now()
    n := 0
    if holes > 0 {
        for range cells.NewMCC().Dance(strings.NewReader(in)).Solutions {
            n++
        }
    } else {
        for range cells.NewXCC().Dance(strings.NewReader(in)).Solutions {
            n++
        }
    }
    return n, time.Since(t0).Round(time.Millisecond)
}
```

28. The frames answer 323 mentions, and Keller's pair of small ones. A 4×5 frame holds five pieces, so two of them together hold all ten; join them side by side and you have a 4×10 rectangle, stack them and you have 5×8, which is why solving that problem solves both rectangles at once.

⟨ Fill the frames 28 ⟩ ≡

```
ps := grow(4)[4]
for _, d := range [][2]int {{10, 4}, {8, 5}, {20, 2}, {21, 2}, {22, 2} }{
    f := rect(0, 0, d[0], d[1])
    ⟨ Count the packings of f 29 ⟩
}
⟨ Pack Keller's two small frames 30 ⟩
```

This code is also defined in section 31.

This code is used in section 3.

29. ⟨ Count the packings of f 29 ⟩ ≡

```
holes := len(f) - 40
in, nopt := problem(f, ps, holes)
n, el := solve(in, holes)
fmt.Printf("%d x %2d: %d cells, %d left empty, %4d options, %7d solutions, %s\n",
    d[1], d[0], len(f), holes, nopt, n, el)
```

This code is used in section 28.

30. ⟨ Pack Keller's two small frames 30 ⟩ ≡

```
f := rect(0, 0, 5, 4)
for c := range rect(20, 0, 5, 4) {
    f[c] = true
}
in, nopt := problem(f, ps, 0)
n, el := solve(in, 0)
fmt.Printf("two 4 x 5 frames: %d options, %d solutions (%d dual pairs), %s\n",
    nopt, n, n/2, el)
```

This code is used in section 28.

31. One more reading of the placement counts, from a direction that knows nothing about symmetries or shifts: every four cells of the frame whose canonical form is one of the ten pieces is a placement of that piece, and nothing else is.

⟨ Fill the frames 28 ⟩ +≡

```
for _, d := range [][2]int {{10, 4}, {21, 2} }{
    f := rect(0, 0, d[0], d[1])
    ⟨ Count the placements a second way 32 ⟩
}
```

32. ⟨ Count the placements a second way 32 ⟩ ≡

```
idx := map[string]int{}
for i, s := range ps {
    idx[s.canon()] = i
}
var cs []cell
for c := range f {
    cs = append(cs, c)
}
brute := map[int]int{}
⟨ Look at every four cells of the frame 33 ⟩
same := true
for i, s := range ps {
    if len(placements(s, f)) ≠ brute[i] {
        same = false
    }
}
fmt.Printf("%d x %2d: the two counts of the placements agree: %v\n",
    d[1], d[0], same)
```

This code is used in section 31.

33. ⟨ Look at every four cells of the frame 33 ⟩ ≡

```
for a := 0; a < len(cs); a++ {
    for b := a + 1; b < len(cs); b++ {
        for c := b + 1; c < len(cs); c++ {
            for e := c + 1; e < len(cs); e++ {
                q := shape{cs[a], cs[b], cs[c], cs[e]}
                if i, ok := idx[q.norm().canon()]; ok {
                    brute[i]++
                }
            }
        }
    }
}
```

This code is used in section 32.

34. The pixel reduction. Part (b) asks for polyskews to be reduced to polyominoes, and answers with a picture: a square becomes a five-pixel cross and a rhombus a three-pixel diagonal, leaning the way the rhombus leans. Cell (x, y) sits at pixel $(2x, 2y)$, and the crosses and diagonals then tile the pixel plane exactly.

The point of the reduction is the claim that "the shapes fit together only when squares and rhombuses alternate properly"—that an ordinary polyomino solver, allowed every shift and not just the even ones, cannot go wrong. This part tests that claim by doing exactly that and comparing.

⟨ Types 4 ⟩ +≡

```
type pix struct { u, v int }
```

35. ⟨ Functions 5 ⟩ +≡

```
func pixels(c cell) []pix {
    u, v := 2 * c.x, 2 * c.y
    k, o := kindOf(c)
    if k ≡ 1 {
        return []pix{{u, v}, {u − 1, v}, {u + 1, v}, {u, v − 1}, {u, v + 1}}
    }
    if o ≡ 0 {
        return []pix{{u − 1, v − 1}, {u, v}, {u + 1, v + 1}}
    }
    return []pix{{u − 1, v + 1}, {u, v}, {u + 1, v − 1}}
}
```

36. ⟨ Functions 5 ⟩ +≡

```
func normPix(p []pix) []pix {
    q := append([]pix{}, p...)
    mu, mv := q[0].u, q[0].v
    for _, x := range q {
        if x.u < mu {
            mu = x.u
        }
        if x.v < mv {
            mv = x.v
        }
    }
    for i := range q {
        q[i].u -= mu
        q[i].v -= mv
    }
    sort.Slice(q, func(i, j int) bool {
        if q[i].v ≠ q[j].v {
            return q[i].v < q[j].v
        }
        return q[i].u < q[j].u
    })
    return q
}
func pixKey(p []pix) string {
    var b strings.Builder
    for _, x := range p {
        fmt.Fprintf(&b, "%d,%d;", x.u, x.v)
    }
    return b.String()
}
```

37. ⟨ Functions 5 ⟩ +≡

```
func pixShape(s shape) []pix {
    var out []pix
    for _, c := range s {
        out = append(out, pixels(c)...)
    }
    return normPix(out)
}
func pixImages(p []pix) [][]pix {
    seen := map[string]bool{}
    var out [][]pix
    for _, m := range mats {
        q := make([]pix, len(p))
        for i, x := range p {
            q[i] = pix{m[0]*x.u + m[1]*x.v, m[2]*x.u + m[3]*x.v}
        }
        q = normPix(q)
        if k := pixKey(q); ¬seen[k] {
            seen[k] = true
            out = append(out, q)
        }
    }
    return out
}
```

38. ⟨ Functions 5 ⟩ +≡

```
func pixProblem(frame map[cell]bool, ps []shape)(string, int) {
    reg := map[pix]bool{}
    for c := range frame {
        for _, x := range pixels(c) {
            reg[x] = true
        }
    }
    var items []string
    for x := range reg {
        items = append(items, pname(x))
    }
    sort.Strings(items)
    var b strings.Builder
    for i := range ps {
        fmt.Fprintf(&b, "p%d␣", i)
    }
    b.WriteString(strings.Join(items, "␣"))
    b.WriteString("\n")
    n := 0
    ⟨ Write an option for every pixel placement 39 ⟩
    return b.String(), n
}
func pname(x pix) string { return fmt.Sprintf("q%03d.%03d", x.u + 100, x.v + 100) }
```

39. ⟨ Write an option for every pixel placement 39 ⟩ ≡

```
for i, s := range ps {
    for _, im := range pixImages(pixShape(s)) {
        for dv := −60; dv ≤ 60; dv++ {
            for du := −60; du ≤ 60; du++ {
                var cs []string
                ok := true
                for _, x := range im {
                    y := pix{x.u + du, x.v + dv}
                    if ¬reg[y] {
                        ok = false
                        break
                    }
                    cs = append(cs, pname(y))
                }
                if ¬ok {
                    continue
                }
                sort.Strings(cs)
                fmt.Fprintf(&b, "p%d␣%s\n", i, strings.Join(cs, "␣"))
                n++
            }
        }
    }
}
```

This code is used in section 38.

40. The two-row frames need their empty cells, and in the pixel picture an empty cell is a whole cross or a whole diagonal—so the two monoskews stand in for the holes.

⟨ Fill the frames through the pixel reduction 40 ⟩ ≡

```
ps := grow(4)[4]
withHoles := append(append([]shape{}, ps...), grow(1)[1]...)
for _, d := range [][2]int {{10, 4}, {8, 5}, {21, 2} }{
    f := rect(0, 0, d[0], d[1])
    set := ps
    if len(f) > 40 {
        set = withHoles
    }
    in, nopt := pixProblem(f, set)
    n, el := solve(in, 0)
    fmt.Printf("%d␣x␣%2d␣through␣the␣pixels:␣%4d␣options,␣%7d␣solutions,␣%s\n",
        d[1], d[0], nopt, n, el)
}
```

This code is used in section 3.

41. Pairing the solutions. Answer 323 says the counts "can be divided by 2, because solutions to this problem come in pairs": changing the spins of a valid solution gives another valid solution, with $K \leftrightarrow L$, $S \leftrightarrow Z$ and $U \leftrightarrow V$ swapped. Sliding the frame by $(1, 1)$ moves it off itself, so the dual of a solution is that slide followed by whichever symmetry of the grid brings the frame back—and the pieces have to be relabelled, each by its own dual.

⟨ Types 4 ⟩ +≡

```
type sol struct { piece [][]cell }
```

42. ⟨ Functions 5 ⟩ +≡

```
func parse(opts [][]string) sol {
    var s sol
    s.piece = make([][]cell, 10)
    for _, o := range opts {
        var idx int
        fmt.Sscanf(o[0], "p%d", &idx)
        var cs []cell
        for _, t := range o[1:] {
            var x, y int
            fmt.Sscanf(t, "c%d.%d", &x, &y)
            cs = append(cs, cell{x - 50, y - 50})
        }
        s.piece[idx] = sorted(cs)
    }
    return s
}
```

43. ⟨ Functions 5 ⟩ +≡

```
func sorted(cs []cell) []cell {
    sort.Slice(cs, func(i, j int) bool {
        if cs[i].y ≠ cs[j].y {
            return cs[i].y < cs[j].y
        }
        return cs[i].x < cs[j].x
    })
    return cs
}
```

44. ⟨ Functions 5 ⟩ +≡

```
func (s sol) key() string {
    var b strings.Builder
    for i, cs := range s.piece {
        fmt.Fprintf(&b, "%d:%s", i, shape(cs).key())
    }
    return b.String()
}
```

45. Forgetting which piece is which leaves the *unskewed arrangement*: the partition of the rectangle into ten tetrominoes. Answer 323 counts those up to the reflections of the rectangle, so the key takes the least over the four of them.

```
⟨ Functions 5 ⟩ +≡
    func (s sol) unskewed(w, h int) string {
        best := ""
        for k := 0; k < 4; k++ {
            var parts []string
            for _, cs := range s.piece {
                var d []cell
                for _, c := range cs {
                    ⟨ Reflect c according to k 46 ⟩
                }
                parts = append(parts, shape(sorted(d)).key())
            }
            sort.Strings(parts)
            if t := strings.Join(parts, "|"); best == "" ∨ t < best {
                best = t
            }
        }
        return best
    }
```

46. ⟨ Reflect c according to k 46 ⟩ ≡

```
    x, y := c.x, c.y
    if k & 1 == 1 {
        x = w - 1 - x
    }
    if k & 2 == 2 {
        y = h - 1 - y
    }
    d = append(d, cell{x, y})
```

This code is used in section 45.

47. ⟨ Pair up the solutions 47 ⟩ ≡

```
ps := grow(4)[4]
perm := make([]int, 10)
byCanon := map[string]int{ }
for i, q := range ps {
    byCanon[q.canon( )] = i
}
for i, q := range ps {
    perm[i] = byCanon[q.dual( ).canon( )]
}
for _, d := range [ ][2]int {{10, 4}, {8, 5} }{
    w, h := d[0], d[1]
    ⟨ Collect the solutions of this frame 48 ⟩
    ⟨ Find the map that takes a solution to its dual 49 ⟩
    ⟨ Say how the solutions pair up and where they come from 51 ⟩
}
```

This code is used in section 3.

48. ⟨ Collect the solutions of this frame 48 ⟩ ≡

```
f := rect(0, 0, w, h)
in, _ := problem(f, ps, 0)
var all [ ]sol
seen := map[string]bool{ }
for so := range cells.NewXCC( ).Dance(strings.NewReader(in)).Solutions {
    var opts [ ][ ]string
    for _, o := range so {
        opts = append(opts, o)
    }
    s := parse(opts)
    all = append(all, s)
    seen[s.key( )] = true
}
```

This code is used in section 47.

49. ⟨ Find the map that takes a solution to its dual 49 ⟩ ≡

```
var move func(cell) cell
for _, g := range symmetries() {
    for b := −20; b ≤ 20 ∧ move ≡ Λ; b++ {
        for a := −20; a ≤ 20 ∧ move ≡ Λ; a++ {
            try := func(c cell) cell {
                e := apply(g.m, cell{c.x + 1, c.y + 1})
                return cell{e.x + g.t.x + 2 ∗ a, e.y + g.t.y + 2 ∗ b}
            }
            ⟨ Take try if it carries the frame to itself 50 ⟩
        }
    }
    if move ≠ Λ {
        break
    }
}
```

This code is used in section 47.

50. ⟨ Take *try* if it carries the frame to itself 50 ⟩ ≡

```
good := true
for c := range f {
    if ¬f[try(c)] {
        good = false
        break
    }
}
if good {
    move = try
}
```

This code is used in section 49.

51. A solution's dual should be another solution of the same frame, and never the solution itself—which is what makes the counts even.

⟨ Say how the solutions pair up and where they come from 51 ⟩ ≡

```
fixed, missing := 0, 0
groups := map[string]int{}
for _, s := range all {
    ⟨ Build the dual of s 52 ⟩
    if t.key() ≡ s.key() {
        fixed++
    }
    if ¬seen[t.key()] {
        missing++
    }
    groups[s.unskewed(w, h)]++
}
hist := map[int]int{}
for _, v := range groups {
    hist[v]++
}
fmt.Printf("%d x %2d: %d solutions, %d self-dual, %d duals not solutions\n",
    h, w, len(all), fixed, missing)
fmt.Printf("      %d unskewed arrangements up to reflection, by size %v\n",
    len(groups), hist)
```

This code is used in section 47.

52. ⟨ Build the dual of s 52 ⟩ ≡

```
var t sol
t.piece = make([][]cell, 10)
for i, cs := range s.piece {
    var d []cell
    for _, c := range cs {
        d = append(d, move(c))
    }
    t.piece[perm[i]] = sorted(d)
}
```

This code is used in section 51.

53. Skewing the arrangements. The last check comes at the counts from the other end. Answer 323 says that an arrangement of ten unskewed tetrominoes—one square, one straight, two skews, two tees and four ells—"can be skewed in four ways, because we have two choices for which cells should be rhombuses and two choices for the spins; and it will be a valid skewed solution if and only if the resulting ten tetraskews are distinct."

The four ways are the four places the rectangle can sit in the grid, modulo 2 in each direction. So: enumerate the arrangements, skew each of them four ways, and count the ones whose ten pieces come out distinct. If the answer is right, each of the four gives the number of solutions of the corresponding frame.

⟨ Functions 5 ⟩ +≡

```
func tetrominoes()(map[string][]cell, []string) {
    return map[string][]cell{
        "square": {{0,0},{1,0},{0,1},{1,1}},
        "straight": {{0,0},{1,0},{2,0},{3,0}},
        "skew": {{0,0},{1,0},{1,1},{2,1}},
        "tee": {{0,0},{1,0},{2,0},{1,1}},
        "ell": {{0,0},{1,0},{2,0},{0,1}},
    }, []string{"square","straight","skew","tee","ell"}
}
var howMany = map[string]int{
    "square": 1, "straight": 1, "skew": 2, "tee": 2, "ell": 4,
}
```

54. These are ordinary polyominoes, so all eight images count and every shift is allowed.

⟨ Functions 5 ⟩ +≡

```
func plainImages(cs []cell) [][]cell {
    seen := map[string]bool{}
    var out [][]cell
    for _, m := range mats {
        q := make([]cell, len(cs))
        for i, c := range cs {
            q[i] = apply(m, c)
        }
        q = sorted(shape(q).norm())
        ⟨ Slide the image hard against the corner 55 ⟩
        if k := shape(q).key(); ¬seen[k] {
            seen[k] = true
            out = append(out, q)
        }
    }
    return out
}
```

55. The polyskew *norm* leaves a shape at 0 or 1 because odd shifts are not symmetries of the skewed grid; an unskewed tetromino has no such scruple.

⟨ Slide the image hard against the corner 55 ⟩ ≡

```
mx, my := q[0].x, q[0].y
for _, c := range q {
    if c.x < mx {
        mx = c.x
    }
    if c.y < my {
        my = c.y
    }
}
for i := range q {
    q[i] = cell{q[i].x − mx, q[i].y − my}
}
q = sorted(q)
```

This code is used in section 54.

56. ⟨ Functions 5 ⟩ +≡

```
func tilingProblem(w, h int) string {
    tets, names := tetrominoes()
    var b strings.Builder
    for _, n := range names {
        fmt.Fprintf(&b, "%d:%d|%s␣", howMany[n], howMany[n], n)
    }
    for y := 0; y < h; y++ {
        for x := 0; x < w; x++ {
            fmt.Fprintf(&b, "r%02d.%02d␣", x, y)
        }
    }
    b.WriteString("\n")
    ⟨ Write an option for every tetromino placement 57 ⟩
    return b.String()
}
```

57. ⟨ Write an option for every tetromino placement 57 ⟩ ≡

```
for _, n := range names {
    for _, im := range plainImages(tets[n]) {
        for dy := 0; dy < h; dy++ {
            for dx := 0; dx < w; dx++ {
                var cs []string
                ok := true
                for _, c := range im {
                    x, y := c.x + dx, c.y + dy
                    if x ≥ w ∨ y ≥ h {
                        ok = false
                        break
                    }
                    cs = append(cs, fmt.Sprintf("r%02d.%02d", x, y))
                }
                if ok {
                    sort.Strings(cs)
                    fmt.Fprintf(&b, "%s␣%s\n", n, strings.Join(cs, "␣"))
                }
            }
        }
    }
}
```

This code is used in section 56.

58. ⟨ Skew every arrangement of ten tetrominoes 58 ⟩ ≡

```
ps := grow(4)[4]
idx := map[string]int{}
for i, q := range ps {
    idx[q.canon()] = i
}
for _, d := range [][2]int {{10, 4}, {8, 5}} {
    w, h := d[0], d[1]
    ⟨ Skew every arrangement of this rectangle 59 ⟩
}
```

This code is used in section 3.

59. ⟨ Skew every arrangement of this rectangle 59 ⟩ ≡

```
valid := map[cell]int{}
hist := map[int]int{}
total, notDual := 0, 0
for so := range cells.NewMCC().Dance(strings.NewReader(tilingProblem(w, h))).Solutions {
    total++
    ⟨ Read the arrangement 60 ⟩
    ⟨ Skew it four ways 61 ⟩
}
fmt.Printf("%d x %2d: %d arrangements; valid skewings by offset %v\n",
    h, w, total, valid)
fmt.Printf("       how many of the four work: %v, and %d of the pairs are not dual\n",
    hist, notDual)
```

This code is used in section 58.

60. ⟨ Read the arrangement 60 ⟩ ≡

```
var tiles [][]cell
for _, o := range so {
    var cs []cell
    for _, t := range o[1:] {
        var x, y int
        fmt.Sscanf(t, "r%d.%d", &x, &y)
        cs = append(cs, cell{x, y})
    }
    tiles = append(tiles, cs)
}
```

This code is used in section 59.

61. ⟨ Skew it four ways 61 ⟩ ≡

```
var here []cell
for y0 := 0; y0 < 2; y0++ {
    for x0 := 0; x0 < 2; x0++ {
        seen := map[int]bool{}
        for _, cs := range tiles {
            var q shape
            for _, c := range cs {
                q = append(q, cell{c.x + x0, c.y + y0})
            }
            if i, ok := idx[q.norm().canon()]; ok {
                seen[i] = true
            }
        }
        if len(seen) ≡ 10 {
            valid[cell{x0, y0}]++
            here = append(here, cell{x0, y0})
        }
    }
}
hist[len(here)]++
if len(here) ≡ 2 {
    if mod2(here[0].x + 1) ≠ here[1].x ∨ mod2(here[0].y + 1) ≠ here[1].y {
        notDual++
    }
}
```

This code is used in section 59.

Polyskews